INFORME TÉCNICO DE AUDITORÍA: INTEGRIDAD DE LA INFORMACIÓN

Demostración de Colisiones MD5 y Verificación Jerárquica con Merkle Trees

---

Asignatura: Seguridad en Sistemas de Información e Internet | **Equipo:** Security Team 2 (ST-2)
Autores: Raimundo García Pérez, Konstantinos Peridis, Luis Alarcón Conradi
Fecha: Septiembre de 2026 | **Código de Entrega:** ST-2-CAI0B

---

1. Resumen Ejecutivo

El presente informe documenta dos comprobaciones prácticas de laboratorio sobre la integridad de los datos digitales y los límites de las funciones resumen en seguridad informática:

1. **Demostración de colisión práctica en MD5:** Mediante la evaluación forense de dos programas ejecutables independientes para Linux (`hello` y `erase`), se certifica la obtención del mismo resumen criptográfico MD5 exacto (`da5c61e1edc0f18337e46418e48c1290`), a pesar de presentar 6 bytes de diferencia en su contenido binario y comportamientos en ejecución radicalmente divergentes (mensaje inofensivo frente a simulación destructiva).
2. **Verificación jerárquica con Merkle Tree:** Se ha diseñado e implementado en Python un árbol binario de Merkle de 8 hojas alimentado con letras derivadas de las identidades del equipo. Al alterar puntualmente la séptima hoja (`S` $\rightarrow$ `U`), la raíz muta de `0e5c37db...` a `430b43c0...`. Se demuestra empíricamente que solo se recalculan **4 de los 15 nodos (26.7%)**, manteniéndose el **73.3% del árbol intacto**.

Veredicto Técnico: Se ratifica la pérdida definitiva de seguridad del algoritmo MD5 para propósitos de verificación de software, firmas y distribución de binarios, y se valida la eficiencia logarítmica $\mathcal{O}(\log_2 N)$ de los árboles de Merkle como pilar de integridad en redes distribuidas y blockchain.

2. Primera Parte: Demostración Práctica de Colisiones en MD5

2.1. Contextualización criptográfica

Una función resumen transforma un mensaje de longitud arbitraria en un compendio de longitud fija (128 bits / 32 dígitos hexadecimales en MD5). Toda función hash criptográfica debe satisfacer idealmente dos directivas: resistencia a preimagen y resistencia a colisiones (imposibilidad práctica de hallar dos mensajes $M_1 \neq M_2$ tales que $H(M_1) = H(M_2)$).

Debido al principio del palomar y la paradoja del cumpleaños, las colisiones matemáticas existen necesariamente. En MD5, el procesamiento iterativo en bloques de 64 bytes (construcción de Merkle-Damgård) presenta debilidades estructurales que permiten forzar trayectorias diferenciales que anulan las divergencias internas entre bloques en tiempo polinomial.

2.2. Entorno y comandos ejecutados

Se evaluaron los binarios desarrollados por Peter Selinger (Universidad de Dalhousie) sobre un sistema Kali Linux. Los comandos de auditoría ejecutados en terminal fueron:

```
chmod +x hello erase
md5sum hello erase
```

```
cmp -l hello erase | wc -l
./hello
./erase
```

2.3. Resultados obtenidos y evidencias forenses

La inspección forense en consola constata la colisión criptográfica y la divergencia en tiempo de ejecución:

Fichero	Propósito aparente	Hash MD5 obtenido	Diferencias	Mensaje en ejecución
`hello`	Demostración benigna	`da5c61e1edc0f18337e46418e48c1290`	Base (0 bytes)	`Hello, world!`
`erase`	Carga simulada hostil	`da5c61e1edc0f18337e46418e48c1290`	6 bytes (`cmp`)	`This program is evil!!! Erasing hard drive...`

Cuadro 1: Contraste de integridad y comportamiento entre los binarios analizados.

2.4. Mecanismo técnico de la colisión

El método implementado se sustenta en tres factores:

1. **Prefijo común:** Ambos ficheros comparten el formato ejecutable ELF y su estructura de inicio procesada en bloques de 64 bytes.
2. **Bloques diferenciales compensados:** Se inyectan dos bloques binarios distintos ($B_1 \neq B_2$) generados matemáticamente de modo que las alteraciones en los registros de compresión de MD5 se compensen exactamente tras dos bloques consecutivos. Al poseer el mismo estado interno, cualquier sufijo idéntico mantiene la igualdad del hash por la propiedad de extensión de Merkle-Damgård.
3. **Bifurcación lógica condicional:** El código máquina compilado es idéntico en ambos programas; sin embargo, incluye una bifurcación (`if/else`) que lee el contenido de su propio array de datos en memoria. Según el byte detectado, el flujo salta a la rutina inofensiva o a la rutina simulada maliciosa.

```c
/* Estructura logica de bifurcacion interna compartida */
if (datos_bloque[OFFSET_DIFERENCIADOR] == VALOR_BENIGNO) {
    printf("Hello, world!\n");
} else {
    printf("This program is evil!!! Erasing hard drive...\n");
}
```

2.5. Riesgos y consecuencias en seguridad corporativa

- **Falsificación en distribución de software:** Un atacante en posición Man-in-the-Middle o con acceso al servidor puede sustituir un instalador legítimo por malware sin alterar el checksum MD5 publicado.
- **Invalidez en verificación de imágenes ISO:** La comprobación de integridad mediante `md5sum` no garantiza la ausencia de puertas traseras.
- **Falsificación de certificados y firmas:** Generación de dos certificados X.509 con idéntico hash pero diferentes claves públicas o permisos.

3. Segunda Parte: Verificación Jerárquica mediante Merkle Tree

3.1. Metodología de selección de letras (Identidades del Security Team)

El equipo de trabajo está formado por tres integrantes: **Konstantinos**, **Luis** y **Ray**. Para garantizar un proceso formal y determinista:

- Se extrajo el conjunto de caracteres presentes en sus identidades: $\{K, O, N, S, T, A, I, L, U, R, Y\}$.
- Se seleccionaron 8 letras únicas ordenadas lexicográficamente para alimentar las 8 hojas del árbol:

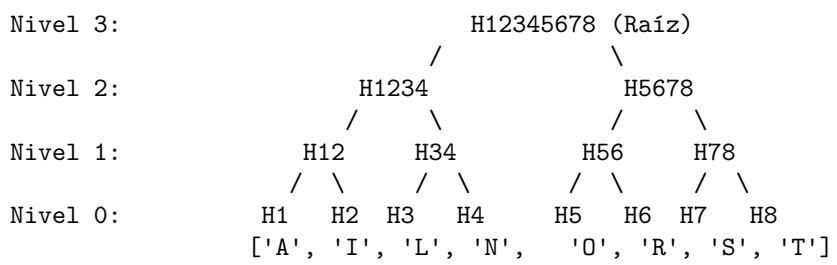
Hoja	Carácter	Integrante de origen	Cálculo de la Hoja ($Nivel\ 0$)
H_1	`'A'`	Konstantinos	MD5("A") = `7fc56270e7a70fa81a5935b72eacbe29`
H_2	`'I'`	Konstantinos	MD5("I") = `dd7536794b63bf90eccfd37f9b147d7f`
H_3	`'L'`	Luis	MD5("L") = `d20caec3b48a1eef164cb4ca81ba2587`
H_4	`'N'`	Konstantinos	MD5("N") = `8d9c307cb7f3c4a32822a51922d1ceaa`
H_5	`'O'`	Konstantinos	MD5("O") = `f186217753c37b9b9f958d906208506e`
H_6	`'R'`	Ray	MD5("R") = `e1e1d3d40573127e9ee0480caf1283d6`
H_7	`'S'`	Konstantinos	MD5("S") = `5dbc98dcc983a70728bd082d1a47546e`
H_8	`'T'`	Konstantinos	MD5("T") = `b9ece18c950afbfa6b0fdbfa4ff731d3`

Cuadro 2: Mapeo de letras seleccionadas y cómputo de hojas base.

3.2. Regla de agregación y topología del árbol

El árbol binario consta de 4 niveles y 15 nodos calculados recursivamente:

$$H_{\text{padre}} = \text{MD5}\big(H_{\text{hijo_izq}} \parallel H_{\text{hijo_der}}\big) \tag{1}$$



3.3. Evaluación del efecto avalancha (Modificación de H_7: 'S' $\rightarrow$ 'U')

Al sustituir en el séptimo nodo la letra 'S' por 'U', su resumen cambia a MD5("U") = 4c614360da93c0a041b22
La siguiente matriz recoge el estado de los 15 nodos tras recalcular el árbol:

Nivel	Nodo	Hash Original (MD5)	Hash Recalculado (MD5)	Estado
0	H_1	7fc56270e7a70fa81a5935b72eacbe29	7fc56270e7a70fa81a5935b72eacbe29	INTACTO
0	H_2	dd7536794b63bf90eccfd37f9b147d7f	dd7536794b63bf90eccfd37f9b147d7f	INTACTO
0	H_3	d20caec3b48a1eef164cb4ca81ba2587	d20caec3b48a1eef164cb4ca81ba2587	INTACTO
0	H_4	8d9c307cb7f3c4a32822a51922d1ceaa	8d9c307cb7f3c4a32822a51922d1ceaa	INTACTO
0	H_5	f186217753c37b9b9f958d906208506e	f186217753c37b9b9f958d906208506e	INTACTO
0	H_6	e1e1d3d40573127e9ee0480caf1283d6	e1e1d3d40573127e9ee0480caf1283d6	INTACTO
0	H_7	5dbc98dcc983a70728bd082d1a47546e	4c614360da93c0a041b22e537de151eb	MODIFICADO
0	H_8	b9ece18c950afbfa6b0fdbfa4ff731d3	b9ece18c950afbfa6b0fdbfa4ff731d3	INTACTO
1	H_{12}	4c03c5ddaa5d79dfb403829882d7afa1	4c03c5ddaa5d79dfb403829882d7afa1	INTACTO
1	H_{34}	94bc49f1be10f290377acdb59476e0bb	94bc49f1be10f290377acdb59476e0bb	INTACTO
1	H_{56}	a18378c73b957f7986e3c7afb346ad47	a18378c73b957f7986e3c7afb346ad47	INTACTO
1	H_{78}	45775e398befd4b39c5ca6e132b8b5bf	87a36d0803334797b7d3482b72131099	MODIFICADO
2	H_{1234}	ebd217460529bdd32d5d907dce30b906	ebd217460529bdd32d5d907dce30b906	INTACTO
2	H_{5678}	e6c1ed52e988db44254e9eb3f801e9e2	1840e769e54ededd30eaa70c9c4362d9	MODIFICADO
3	Raíz	0e5c37db70512ff1e74b743d24d9024d	430b43c05f6752cf1b30354b381db387	MODIFICADO

Cuadro 3: Evaluación de propagación del cambio: comparación nodo a nodo.

3.4. Resúmenes recalculados y eficiencia logarítmica

Los hashes que obligatoriamente deben volver a computarse se limitan a la ruta ascendente directa (*Merkle Path*):

1. H_7: Por alteración del dato origen ('S' $\rightarrow$ 'U').
2. H_{78}: Requiere calcular MD5($H_7^{\text{nuevo}} \parallel H_8^{\text{intacto}}$).
3. H_{5678}: Requiere calcular MD5($H_{56}^{\text{intacto}} \parallel H_{78}^{\text{nuevo}}$).
4. Raíz $H_{12345678}$: Requiere calcular MD5($H_{1234}^{\text{intacto}} \parallel H_{5678}^{\text{nuevo}}$).

Balance de Eficiencia:

- Nodos recalculados: **4 de 15 (26.7%)**.
- Nodos no alterados: **11 de 15 (73.3%)**.
- En conjuntos masivos de N bloques, para auditar una modificación solo se transmiten $\log_2(N)$ hashes hermanos. En Bitcoin o Ethereum, esto permite a los clientes ligeros (SPV) validar transacciones sin descargar la cadena de bloques íntegra.

4. Conclusiones Técnicas y Recomendaciones

4.1. Conclusiones

1. **Inseguridad manifiesta de MD5:** La existencia de colisiones prácticas invalida MD5 para cualquier control de integridad, firma digital o control de versiones seguro.
2. **Falso sentido de seguridad por igualdad de resumen:** La coincidencia en un hash MD5 no certifica identidad de contenidos ni ausencia de código malicioso.
3. **Escalabilidad de los árboles de Merkle:** Permiten vincular criptográficamente miles de elementos a una única raíz de confianza compacta de 128 o 256 bits, auditando alteraciones con coste $\mathcal{O}(\log_2 N)$.

4.2. Recomendaciones

1. **Migración a la familia SHA-2 / SHA-3:** Reemplazar MD5 por SHA-256 o SHA-512 en el almacenamiento y verificación de binarios corporativos.
2. **Despliegue de sistemas de monitorización FIM:** Utilizar utilidades especializadas como Tripwire o VeriSys para detectar alteraciones no autorizadas en ficheros del sistema y repositorios.
3. **Firma digital con infraestructura PKI:** Toda entrega de software debe acompañarse de una firma digital asimétrica que garantice integridad y autenticidad de origen.

5. Propuesta de Consultoría: SecurTech Advisory

SecurTech Advisory ofrece a la entidad un catálogo de servicios para modernizar sus garantías de integridad:

- **Auditoría de Integridad y Arquitectura Criptográfica:** Localización y reemplazo de funciones obsoletas (MD5, SHA-1) en procesos de distribución y APIs.
- **Implantación de Soluciones FIM:** Despliegue de agentes centralizados para monitorización de integridad en servidores de misión crítica y bases de datos.
- **Capacitación Técnica en Cadena de Suministro:** Formación sobre verificación criptográfica, seguridad en repositorios Git y arquitecturas Merkle.

6. Bibliografía

- Ceballos Guerrero, Rafael. *Tema 2: Integridad de la Información (Partes 1 y 2)*. Dpto. Lenguajes y Sistemas Informáticos, Universidad de Sevilla.
- Selinger, Peter (2006). *MD5 Collision Demo*. Dalhousie University. https://www.mscs.dal.ca/~selinger/md5collision/
- Wang, X., & Yu, H. (2005). *How to Break MD5 and Other Hash Functions*. EUROCRYPT 2005.
- Merkle, Ralph C. (1987). *A Digital Signature Based on a Conventional Encryption Function*. CRYPTO '87.
- Python Software Foundation. *hashlib – Secure hashes and message digests* (Python 3 Documentation).

7. Relación de Anexos

- **Anexo I:** Script de cómputo del Árbol de Merkle (`arbol.py`).
- **Anexo II:** Evidencias de consola de ejecución en Kali Linux (`sol-ej1.txt` y `sol-ej2.txt`).

Anexo I: Código Fuente del Generador Merkle (`arbol.py`)

El siguiente script en Python calcula los niveles del árbol, aplica la sustitución en la hoja 7 y computa las diferencias de forma jerárquica:

```
import hashlib

def md5_hash(data: str) -> str:
    return hashlib.md5(data.encode('utf-8')).hexdigest()

def merkle_levels(letters: list[str]) -> list[list[str]]:
    if len(letters) != 8:
        raise ValueError("Se requieren exactamente 8 letras.")

    levels = [[md5_hash(char) for char in letters]]
    while len(levels[-1]) > 1:
        cur = levels[-1]
        levels.append([
            md5_hash(cur[i] + cur[i + 1])
            for i in range(0, len(cur), 2)
        ])
    return levels

def print_tree(letters: list[str], title: str) -> list[list[str]]:
    print("=" * 80)
    print(f" {title.upper()}")
    print("=" * 80)
    print(f"Entrada (Hojas 1-8): {'  '.join(letters)}\n")

    levels = merkle_levels(letters)
    labels = ["Hojas (Nivel 0)", "Nivel 1 (Pares)", "Nivel 2 (Subramas)", "Raiz (Nivel
        3)"]

    for lvl_idx, lvl_hashes in enumerate(levels):
        print(f"[*] {labels[lvl_idx]}:")
        for i, h in enumerate(lvl_hashes):
            print(f"    - Nodo [{lvl_idx}][{i}]: {h}")
        print()

    print(f"[>>>] RAIZ DE MERKLE: {levels[-1][0]}\n")
    return levels

def main():
    letras_orig = ['A', 'I', 'L', 'N', 'O', 'R', 'S', 'T']
    arbol_orig = print_tree(letras_orig, "Arbol de Merkle Original")

    letras_mod = ['A', 'I', 'L', 'N', 'O', 'R', 'U', 'T']
    arbol_mod = print_tree(letras_mod, "Arbol de Merkle Modificado (Hoja 7: 'S' -> 'U
        ')")

if __name__ == "__main__":
    main()
```